Dokumentasi Resmi & Sintaks PHP (8.2/8.3/8.4 Up-to-Date)

Bahasa: PHP | Total Chunk/Dokumen: 95 | Disiapkan untuk RAG System

[php_001] PHP - Basic Syntax & Types

PHP Basic Syntax (Tags, Escaping, Comments): PHP tags

Sumber: ht

PHP tags
When PHP processes a file, it recognizes the opening and closing
tags, <?php and ?> , to
define the boundaries of PHP code execution. Content outside these
tags is ignored by the PHP parser, allowing PHP to seamlessly embed
in various document types.
A whitespace character (space, tab, or newline) must follow
<?php to ensure proper token separation.
Omitting this whitespace will result in a syntax error.
PHP also includes the short echo tag <?= ,
which is shorthand for <?php echo .
PHP Opening and Closing Tags
Short tags (the third line in the example above) are available by default but can be disabled
either via the short_open_tag
&php.ini; configuration file directive, or are disabled by default if
PHP is built with the --disable-short-tags configuration.
As short tags can be disabled it is recommended to only use the normal
tags (<?php ?> and <?= ?>) to
maximise compatibility.
If a file ends with PHP code, it is preferable to omit the PHP closing tag
at the end of the file. This prevents accidental whitespace or new lines
being added after the PHP closing tag, which may cause unwanted effects
because PHP will start output buffering when there is no intention from
the programmer to send any output at that point in the script.
PHP Code Only File

Contoh Kode / Perintah CLI:

```
2.  You can use the short echo tag to .
    It's equivalent to .

3.
```

[php_002] PHP - Basic Syntax & Types

PHP Basic Syntax (Tags, Escaping, Comments): Escaping from HTML

Sumber: ht

Escaping from HTML
Everything outside of a pair of opening and closing tags is ignored by the
PHP parser which allows PHP files to have mixed content. This allows PHP
to be embedded in HTML documents, for example to create templates.
Embedding PHP in HTML
This works as expected, because when the PHP interpreter hits the ?> closing
tags, it simply starts outputting whatever it finds (except for the
immediately following newline - see
instruction separation)

until it hits another opening tag unless in the middle of a conditional
statement in which case the interpreter will determine the outcome of the
conditional before making a decision of what to skip over.
See the next example.
Using structures with conditions
Advanced escaping using conditions
In this example PHP will skip the blocks where the condition is not met, even
though they are outside of the PHP open/close tags; PHP skips them according
to the condition since the PHP interpreter will jump over blocks contained
within a condition that is not met.
For outputting large blocks of text, dropping out of PHP parsing mode is
generally more efficient than sending all of the text through
echo or print .
If PHP is embedded within XML or XHTML the normal PHP
<?php ?> must be used to remain compliant
with the standards.

Contoh Kode / Perintah CLI:

```
This is going to be ignored by PHP and displayed by the browser.

This will also be ignored by PHP and displayed by the browser.

// ----------------------------------------

This will show if the expression is true.

  Otherwise this will show.
```

[php_003] PHP - Basic Syntax & Types

PHP Basic Syntax (Tags, Escaping, Comments): Instruction separation

Sumber: ht

Instruction separation
As in C or Perl, PHP requires instructions to be terminated
with a semicolon at the end of each statement. The closing tag
of a block of PHP code automatically implies a semicolon; you
do not need to have a semicolon terminating the last line of a
PHP block. The closing tag for the block will include the immediately
trailing newline if one is present.
Example showing the closing tag encompassing the trailing newline
&example.outputs;
Examples of entering and exiting the PHP parser
The closing tag of a PHP block at the end of a file is optional,
and in some cases omitting it is helpful when using include
or require , so unwanted whitespace will
not occur at the end of files, and you will still be able to add
headers to the response later. It is also handy if you use output
buffering, and would not like to see added unwanted whitespace
at the end of the parts generated by the included files.

Contoh Kode / Perintah CLI:

```
No newline
```

[php_004] PHP - Basic Syntax & Types

PHP Basic Syntax (Tags, Escaping, Comments): Comments

Sumber: ht

Comments
PHP supports 'C', 'C++' and Unix shell-style (Perl style) comments. For example:
Comments
The "one-line" comment styles only comment to the end of
the line or the current block of PHP code, whichever comes first.
This means that HTML code after // ... ?>
or # ... ?> WILL be printed:
?> breaks out of PHP mode and returns to HTML mode, and
// or # cannot influence that.
One Line Comments
'C' style comments end at the first */ encountered.
Make sure you don't nest 'C' style comments. It is easy to make this
mistake if you are trying to comment out a large block of code.

Contoh Kode / Perintah CLI:

```
This is an  example
The header above will say 'This is an  example'.
```

[php_005] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Type declarations

Sumber: ht

Type declarations
Type declarations can be added to function arguments, return values,
as of PHP 7.4.0, class properties, and as of PHP 8.3.0, class constants.
They ensure that the value is of the specified type at call time,
otherwise a TypeError is thrown.
Every single type that PHP supports, with the exception of
resource , can be used within a user-land type declaration.
This page contains a changelog of availability of the different types
and documentation about usage of them in type declarations.
When a class implements an interface method or reimplements a method which
has already been defined by a parent class, it has to be compatible with the
aforementioned definition.
A method is compatible if it follows the
variance rules.

[php_006] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Overview

Sumber: ht

&reftitle.changelog;
&Version;
&Description;
8.3.0
Support for class, interface, trait, and enum constant typing has been added.
8.2.0
Support for DNF types has been added.

8.2.0
Support for the literal type true has been added.
8.2.0
The types null and false can now be used standalone.
8.1.0
Support for intersection types has been added.
8.1.0
Returning by reference from a void function is now deprecated.
8.1.0
Support for the return only type never has been added.
8.0.0
Support for mixed has been added.
8.0.0
Support for the return only type static has been added.
8.0.0
Support for union types has been added.
7.4.0
Support for class properties typing has been added.
7.2.0
Support for object has been added.
7.1.0
Support for iterable has been added.
7.1.0
Support for void has been added.
7.1.0
Support for nullable types has been added.

[php_007] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Atomic Types Usage Notes

Sumber: ht

Atomic Types Usage Notes
Atomic types have straightforward behaviour with some minor caveats which
are described in this section.

[php_008] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Scalar types

Sumber: ht

Scalar types
Name aliases for scalar types (bool , int ,
float , string) are not supported.
Instead, they are treated as class or interface names.
For example, using boolean as a type declaration
will require the value to be an &instanceof; the class or interface
boolean , rather than of type bool :
&example.outputs.8;

[php_009] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): void

Sumber: ht

void
Returning by reference from a void function is deprecated as of PHP 8.1.0,
because such a function is contradictory.
Previously, it already emitted the following
E_NOTICE when called:
Only variable references should be returned by reference .

[php_010] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Callable types

Sumber: ht

Callable types
This type cannot be used as a class property type declaration.
It is not possible to specify the signature of the function.

[php_011] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Type declarations on pass-by-reference Parameters

Sumber: ht

Type declarations on pass-by-reference Parameters
If a pass-by-reference parameter has a type declaration, the type of the
variable is only checked on function entry, at the
beginning of the call, but not when the function returns.
This means that a function can change the type of the variable
passed by reference.
Typed pass-by-reference Parameters
&example.outputs.similar;

[php_012] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Composite Types Usage Notes

Sumber: ht

Composite Types Usage Notes
Composite type declarations are subject to a couple of restrictions and
will perform a redundancy check at compile time to prevent simple bugs.
Prior to PHP 8.2.0, and the introduction of DNF types,
it was not possible to combine intersection types with union types.

[php_013] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Union types

Sumber: ht

Union types
It is not possible to combine the two singleton types false and true together in a union type.
Use bool instead.
Prior to PHP 8.2.0, as false and null could not be used as standalone types, a union type comprised of only these types was not permitted. This comprises the following types: false , false|null , and ?false .
Nullable type syntactic sugar
A single base type declaration can be marked nullable by prefixing the type with a question mark (?).
Thus ?T and T|null are identical.
This syntax is supported as of PHP 7.1.0, and predates generalized union types support.
It is also possible to achieve nullable arguments by making &null; the default value.
This is not recommended because if the default value is changed in a child class a type compatibility violation will be raised as the null type will need to be added to the type declaration.
This behavior is also deprecated since PHP 8.4.
Old way to make arguments nullable
&example.outputs;

[php_014] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Duplicate and redundant types

Sumber: ht

Duplicate and redundant types
To catch simple bugs in composite type declarations, redundant types that can be detected without performing class loading will result in a compile-time error. This includes:
Each name-resolved type may only occur once. Types such as int|string|INT or Countable&Traversable&COUNTABLE result in an error.
Using mixed or never results in an error.
For union types:
If bool is used, false or true cannot be used additionally.
If object is used, class types cannot be used additionally.
If iterable is used, array and Traversable cannot be used additionally.
For intersection types:
Using a type which is not a class-type results in an error.
Using either self , parent , or

static results in an error.
For DNF types:
If a more generic type is used, the more restrictive one is redundant.
Using two identical intersection types.
This does not guarantee that the type is "minimal", because doing so would require loading all used class types.
For example, if A and B are class aliases, then A|B remains a legal union type, even though it could be reduced to either A or B .
Similarly, if class B extends A {} , then A|B is also a legal union type, even though it could be reduced to just A .

[php_015] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Basic class type declaration

Sumber: ht

&reftitle.examples;
Basic class type declaration
&example.outputs.8;
Basic interface type declaration
&example.outputs.8;
Basic return type declaration
&example.outputs;
Returning an object
&example.outputs;
Nullable argument type declaration
&example.outputs;
Nullable return type declaration
Class property type declaration

Contoh Kode / Perintah CLI:

```
id = $id;
        $this->username = $username;
    }
}
?>
```

[php_016] PHP - Type Declarations & DNF Types

Type Declarations (Return Types, Union Types, Intersection Types, DNF Types): Strict typing

Sumber: ht

Strict typing
By default, PHP will coerce values of the wrong type into the expected scalar type declaration if possible. For example, a function that is given an int for a parameter that expects a string will get a variable of type string .
It is possible to enable strict mode on a per-file basis. In strict mode, only a value corresponding exactly to the type declaration will be accepted, otherwise a TypeError will be thrown.
The only exception to this rule is that an int value will

pass a float type declaration.
Function calls from within internal functions will not be affected by the strict_types declaration.
To enable strict mode, the &declare; statement is used with the strict_types declaration:
Strict typing applies to function calls made from within the file with strict typing enabled, not to the functions declared within that file. If a file without strict typing enabled makes a call to a function that was defined in a file with strict typing, the caller's preference (coercive typing) will be respected, and the value will be coerced.
Strict typing is only defined for scalar type declarations.
Strict typing for arguments values
&example.outputs.8;
Coercive typing for argument values
&example.outputs;
Strict typing for return values
&example.outputs;

[php_017] PHP - Control Structures - match Expression

The match Expression (Pattern comparison, Exhaustive evaluation): match

Sumber: ht

match
The match expression branches evaluation based on an identity check of a value.
Similarly to a switch statement, a match expression has a subject expression that is compared against multiple alternatives. Unlike switch , it will evaluate to a value much like ternary expressions.
Unlike switch , the comparison is an identity check (===) rather than a weak equality check (==).
Match expressions are available as of PHP 8.0.0.
Structure of a match expression
Basic match usage
&example.outputs;
Example of using match with comparison operators
&example.outputs;
The result of a match expression does not need to be used.
When a match expression is used as a standalone expression, it must be terminated by a semicolon ; .
The match expression is similar to a switch statement but has some key differences:
A match arm compares values strictly (===) instead of loosely as the switch statement does.
A match expression returns a value.
match arms do not fall through to later cases the way switch statements do.
A match expression must be exhaustive.
Like switch statements, match expressions are executed match arm by match arm.

In the beginning, no code is executed.
The conditional expressions are only evaluated if all previous conditional expressions failed to match the subject expression.
Only the return expression corresponding to the matching conditional expression will be evaluated.
For example:
match expression arms may contain multiple expressions separated by a comma. That is a logical OR, and is a shorthand for multiple match arms with the same right-hand side.
A special case is the default pattern.
This pattern matches anything that wasn't previously matched.
For example:
Multiple default patterns will raise an E_FATAL_ERROR er

Contoh Kode / Perintah CLI:

```
return_expression,
    conditional_expression1, conditional_expression2 => return_expression,
};

// ----------------------------------------

'This food is an apple',
    'bar' => 'This food is a bar',
    'cake' => 'This food is a cake',
};

var_dump($return_value);

// ----------------------------------------

"Baby",
    $age  "Child",
    $age  "Teenager",
    $age >= 40 => "Adult",
    $age > 19 => "Young adult",
};

var_dump($output);
```

[php_018] PHP - Control Structures - match Expression

The match Expression (Pattern comparison, Exhaustive evaluation): Using match expressions to handle non-identity checks

Sumber: ht

Using match expressions to handle non-identity checks
It is possible to use a match expression to handle non-identity conditional cases by using true as the subject expression.
Using a generalized match expression to branch on integer ranges
&example.outputs;
Using a generalized match expression to branch on string content
&example.outputs;

Contoh Kode / Perintah CLI:

```
= 65 => 'senior',
    $age >= 25 => 'adult',
```

```
    $age >= 18 => 'young adult',
    default => 'kid',
};

var_dump($result);

// ----------------------------------------

'en',
    str_contains($text, 'Bienvenue'), str_contains($text, 'Bonjour') => 'fr',
    // ...
};

var_dump($result);
```

[php_019] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): class

Sumber: ht

class
Basic class definitions begin with the
keyword class , followed by a class name,
followed by a pair of curly braces which enclose the definitions
of the properties and methods belonging to the class.
The class name can be any valid label, provided it is not a
PHP reserved word .
As of PHP 8.4.0, using a single underscore _ as a
class name is deprecated.
A valid class name starts with a letter or underscore,
followed by any number of letters, numbers, or underscores.
As a regular expression, it would be expressed thus:
^[a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*$.
A class may contain its
own constants , variables
(called "properties"), and functions (called "methods").
Simple Class definition
The pseudo-variable $this is available when a
method is called from within an object context.
$this is the value of the calling object.
Calling a non-static method statically throws an
Error .
Prior to PHP 8.0.0, this would generate a deprecation notice,
and $this would be undefined.
Some examples of the $this pseudo-variable
&example.outputs.7;
&example.outputs.8;

Contoh Kode / Perintah CLI:

```
var;
    }
}
?>

// ----------------------------------------

foo();
```

```
A::foo();

$b = new B();
$b->bar();

B::bar();
?>
```

[php_020] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): Readonly classes

Sumber: ht

Readonly classes
As of PHP 8.2.0, a class can be marked with the
readonly modifier.
Marking a class as readonly will add the
readonly modifier
to every declared property, and prevent the creation of
dynamic properties .
Moreover, it is impossible to add support for them by using the
AllowDynamicProperties attribute. Attempting to do so
will trigger a compile-time error.
As neither untyped nor static properties can be marked with the
readonly modifier, readonly classes cannot declare
them either:
A readonly class can be
extended
if, and only if, the child class is also a
readonly class.

[php_021] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): new

Sumber: ht

new
To create an instance of a class, the new keyword must
be used. An object will always be created unless the object has a
constructor defined that throws an
Exception on error. Classes
should be defined before instantiation (and in some cases this is a
requirement).
If a variable containing a string with the name of a class is used with
new , a new instance of that class will be created. If
the class is in a namespace, its fully qualified name must be used when
doing this.
If there are no arguments to be passed to the class's constructor,
parentheses after the class name may be omitted.
Creating an instance
As of PHP 8.0.0, using new with arbitrary expressions
is supported. This allows more complex instantiation if the expression
produces a string . The expressions must be wrapped in parentheses.
Creating an instance using an arbitrary expression
In the given example we show multiple examples of valid arbitrary expressions that produce a class name.

This shows a call to a function, string concatenation, and the ::class constant.
&example.outputs.8;
In the class context, it is possible to create a new object by
new self and new parent .
When assigning an already created instance of a class to a new variable, the new variable
will access the same instance as the object that was assigned. This
behaviour is the same when passing instances to a function. A copy
of an already created object can be made by
cloning it.
Object Assignment
&example.outputs;
string(30) "$assigned will have this value"
}
]]>
It's possible to create instances of an object in a couple of ways:
Creating new objects
&example.outputs;
It is possible to access a member of a newly created object
in a single expression:
Access member of newly created object
&example.outputs.similar;
Prior

Contoh Kode / Perintah CLI:

```
var = '$assigned will have this value';

$instance = null; // $instance and $reference become null

var_dump($instance);
var_dump($reference);
var_dump($assigned);
?>

// ----------------------------------------

format('Y'), PHP_EOL;

// surrounding parentheses are optional as of PHP 8.4.0
echo new DateTime()->format('Y'), PHP_EOL;
?>
```

[php_022] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): Properties and methods

Sumber: ht

Properties and methods
Class properties and methods live in separate "namespaces", so it is
possible to have a property and a method with the same name. Referring to
both a property and a method has the same notation, and whether a property
will be accessed or a method will be called, solely depends on the context,
i.e. whether the usage is a variable access or a function call.
Property access vs. method call
&example.outputs;
That means that calling an anonymous

function which has been assigned to a property is not directly
possible. Instead the property has to be assigned to a variable first, for
instance. It is possible to call such a property directly
by enclosing it in parentheses.
Calling an anonymous function stored in a property
&example.outputs;

Contoh Kode / Perintah CLI:

```
bar, PHP_EOL, $obj->bar(), PHP_EOL;

// ----------------------------------------

bar = function() {
            return 42;
        };
    }
}

$obj = new Foo();

echo ($obj->bar)(), PHP_EOL;
```

[php_023] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): extends

Sumber: ht

extends
A class can inherit the constants, methods, and properties of another class by
using the keyword extends in the class
declaration. It is not possible to extend multiple classes; a
class can only inherit from one base class.
The inherited constants, methods, and properties can be overridden by
redeclaring them with the same name defined in the parent
class. However, if the parent class has defined a method or constant
as final ,
they may not be overridden. It is possible to access the overridden
methods or static properties by referencing them
with parent:: .
As of PHP 8.1.0, constants may be declared as final.
Simple Class Inheritance
&example.outputs;

Contoh Kode / Perintah CLI:

```
displayVar();
?>
```

[php_024] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): Signature compatibility rules

Sumber: ht

Signature compatibility rules
When overriding a method, its signature must be compatible with the parent
method. Otherwise, a fatal error is emitted, or, prior to PHP 8.0.0, an
E_WARNING level error is generated.
A signature is compatible if it respects the

variance rules, makes a
mandatory parameter optional, adds only optional new parameters and
doesn't restrict but only relaxes the visibility.
This is known as the Liskov Substitution Principle, or LSP for short.
The constructor ,
and private methods are exempt from these signature
compatibility rules, and thus won't emit a fatal error in case of a
signature mismatch.
Compatible child methods
&example.outputs;
The following examples demonstrate that a child method which removes a parameter, or makes an optional parameter mandatory, is not compatible with the parent method.
Fatal error when a child method removes a parameter
&example.outputs.8.similar;
Fatal error when a child method makes an optional parameter mandatory
&example.outputs.8.similar;
Renaming a method's parameter in a child class is not a signature
incompatibility. However, this is discouraged as it will result in a
runtime Error if
named arguments
are used.
Error when using named arguments and parameters were renamed in a child class
&example.outputs.similar;

Contoh Kode / Perintah CLI:

```
foo();
$extended2 = new Extend2();
$extended2->foo(1);

// ----------------------------------------

test(foo: "foo", bar: "bar"); // ERROR!
```

[php_025] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): ::class

Sumber: ht

::class
The class keyword is also used for class
name resolution.
To obtain the fully qualified name of a class ClassName
use ClassName::class . This is particularly useful with
namespaced classes.
Class name resolution
&example.outputs;
The class name resolution using ::class is a
compile time transformation. That means at the time the class name string
is created no autoloading has happened yet. As a consequence, class names
are expanded even if the class does not exist. No error is issued in
that case.
Missing class name resolution
&example.outputs;
As of PHP 8.0.0, ::class may also be used on
objects. This resolution happens at runtime, not compile time. Its effect is

the same as calling get_class on the object.
Object name resolution
&example.outputs;

[php_026] PHP - Modern OOP - Classes & Objects

Classes and Objects Basics (class, new, extends, readonly classes): Nullsafe methods and properties

Sumber: ht

Nullsafe methods and properties
As of PHP 8.0.0, properties and methods may also be accessed with the "nullsafe" operator instead: ?-> . The nullsafe operator works the same as property or method access as above, except that if the object being dereferenced is &null; then &null; will be returned rather than an exception thrown. If the dereference is part of a chain, the rest of the chain is skipped.
The effect is similar to wrapping each access in an is_null check first, but more compact.
Nullsafe Operator
The nullsafe operator is best used when null is considered a valid and expected possible value for a property or method return. For indicating an error, a thrown exception is preferable.

Contoh Kode / Perintah CLI:

```
getUser(5)?->name;

// Is equivalent to the following code block:
if (is_null($repository)) {
    $result = null;
} else {
    $user = $repository->getUser(5);
    if (is_null($user)) {
        $result = null;
    } else {
        $result = $user->name;
    }
}
?>
```

[php_027] PHP - Modern OOP - Visibility & Properties

Property and Method Visibility (public, private, protected, readonly, asymmetric): Visibility

Sumber: ht

Visibility
The visibility of a property, a method or (as of PHP 7.1.0) a constant can be defined by prefixing the declaration with the keywords public , protected or private . Class members declared public can be accessed everywhere. Members declared protected can be accessed only within the class itself and by inheriting and parent classes. Members declared as private may only be accessed by the class that defines the member.

[php_028] PHP - Modern OOP - Visibility & Properties

Property and Method Visibility (public, private, protected, readonly, asymmetric): Property Visibility

Sumber: ht

Property Visibility
Class properties may be defined as public, private, or protected. Properties declared without any explicit visibility keyword are defined as public.
Property declaration

Contoh Kode / Perintah CLI:

```
public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj = new MyClass();
echo $obj->public; // Works
echo $obj->protected; // Fatal Error
echo $obj->private; // Fatal Error
$obj->printHello(); // Shows Public, Protected and Private


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // We can redeclare the public and protected properties, but not private
    public $public = 'Public2';
    protected $protected = 'Protected2';

    function printHello()
    {
        echo $this->public;
        echo $this->protected;
        echo $this->private;
    }
}

$obj2 = new MyClass2();
echo $obj2->public; // Works
echo $obj2->protected; // Fatal Error
echo $obj2->private; // Undefined
$obj2->printHello(); // Shows Public2, Protected2, Undefined

?>
```

[php_029] PHP - Modern OOP - Visibility & Properties

Property and Method Visibility (public, private, protected, readonly, asymmetric): Asymmetric Property Visibility

Sumber: ht

Asymmetric Property Visibility
As of PHP 8.4, object properties may also have their visibility set asymmetrically, with a different scope for reading (get) and writing (set).

Specifically, the set visibility may be
specified separately, provided it is not more permissive than the
default visibility.
Asymmetric Property visibility
As of PHP 8.5, set visibility may also be applied to static properties of classes.
Asymmetric Static Property Visibility
&example.outputs;
There are a few caveats regarding asymmetric visibility:
Only typed properties may have a separate set visibility.
The set visibility must be the same
as get or more restrictive. That is,
public protected(set) and protected protected(set)
are allowed, but protected public(set) will cause a syntax error.
If a property is public , then the main visibility may be
omitted. That is, public private(set) and private(set)
will have the same result.
A property with private(set) visibility
is automatically final , and may not be redeclared in a child class.
Obtaining a reference to a property follows set visibility, not get .
That is because a reference may be used to modify the property value.
Similarly, trying to write to an array property involves both a get and
set operation internally, and therefore will follow the set
visibility, as that is always the more restrictive.
Spaces are not allowed in the set-visibility declaration.
private(set) is correct.
private(set) is not correct and will result in a parse error.
When a class extends another, the child class may redefine
any property that is not final . When doing so,
it may widen either the main visibility or the set
visibility, provided that the new visibility is the same or wider
than the parent class. However, be aware

Contoh Kode / Perintah CLI:

```
author = $author; // OK
        $this->pubYear = $year; // Fatal Error
    }
}

$b = new Book('How to PHP', 'Peter H. Peterson', 2024);

echo $b->title; // Works
echo $b->author; // Works
echo $b->pubYear; // Fatal Error

$b->title = 'How not to PHP'; // Fatal Error
$b->author = 'Pedro H. Peterson'; // Fatal Error
$b->pubYear = 2023; // Fatal Error
?>

// ----------------------------------------

doAThing(); // Works
echo Manager::$calls; // Works
Manager::$calls = 5; // Fatal error
?>
```

[php_030] PHP - Modern OOP - Visibility & Properties

Property and Method Visibility (public, private, protected, readonly, asymmetric): Method Visibility

Sumber: ht

Method Visibility
Class methods may be defined as public, private, or protected. Methods declared without any explicit visibility keyword are defined as public.
Method Declaration

Contoh Kode / Perintah CLI:

```
MyPublic();
        $this->MyProtected();
        $this->MyPrivate();
    }
}

$myclass = new MyClass;
$myclass->MyPublic(); // Works
$myclass->MyProtected(); // Fatal Error
$myclass->MyPrivate(); // Fatal Error
$myclass->Foo(); // Public, Protected and Private work


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // This is public
    function Foo2()
    {
        $this->MyPublic();
        $this->MyProtected();
        $this->MyPrivate(); // Fatal Error
    }
}

$myclass2 = new MyClass2;
$myclass2->MyPublic(); // Works
$myclass2->Foo2(); // Public and Protected work, not Private

class Bar
{
    public function test() {
        $this->testPrivate();
        $this->testPublic();
    }

    public function testPublic() {
        echo "Bar::testPublic\n";
    }

    private function testPrivate() {
        echo "Bar::testPrivate\n";
    }
}

class Foo extends Bar
{
```

```
    public function testPublic() {
        echo "Foo::testPublic\n";
    }

    private function testPrivate() {
        echo "Foo::testPrivate\n";
    }
}

$myFoo = new Foo();
$myFoo->test(); // Bar::testPrivate
                // Foo::testPublic
?>
```

[php_031] PHP - Modern OOP - Visibility & Properties

Property and Method Visibility (public, private, protected, readonly, asymmetric): Constant Visibility

Sumber: ht

Constant Visibility
As of PHP 7.1.0, class constants may be defined as public, private, or protected. Constants declared without any explicit visibility keyword are defined as public.
Constant Declaration as of PHP 7.1.0

Contoh Kode / Perintah CLI:

```
foo(); // Public, Protected and Private work


/**
 * Define MyClass2
 */
class MyClass2 extends MyClass
{
    // This is public
    function foo2()
    {
        echo self::MY_PUBLIC;
        echo self::MY_PROTECTED;
        echo self::MY_PRIVATE; // Fatal Error
    }
}

$myclass2 = new MyClass2;
echo MyClass2::MY_PUBLIC; // Works
$myclass2->foo2(); // Public and Protected work, not Private
?>
```

[php_032] PHP - Modern OOP - Visibility & Properties

Property and Method Visibility (public, private, protected, readonly, asymmetric): Visibility from other objects

Sumber: ht

Visibility from other objects
Objects of the same type will have access to each others private and protected members even though they are not the same instances. This is because the implementation specific details are already known when inside those objects.

Accessing private members of the same object type
&example.outputs;

Contoh Kode / Perintah CLI:

```
foo = $foo;
    }

    private function bar()
    {
        echo 'Accessed the private method.';
    }

    public function baz(Test $other)
    {
        // We can change the private property:
        $other->foo = 'hello';
        var_dump($other->foo);

        // We can also call the private method:
        $other->bar();
    }
}

$test = new Test('test');

$test->baz(new Test('other'));
?>
```

[php_033] PHP - Modern OOP - Enumerations (Enums)

Enumerations (Pure enums, Backed enums, Methods in enums): Basic Enumerations

Sumber: ht

Basic Enumerations
Enumerations are a restricting layer on top of classes and class constants,
intended to provide a way to define a closed set of possible values for a type.
For a full discussion, see the
Enumerations chapter.

[php_034] PHP - Modern OOP - Enumerations (Enums)

Enumerations (Pure enums, Backed enums, Methods in enums): Casting

Sumber: ht

Casting
If an enum is converted to an object , it is not
modified. If an enum is converted to an array ,
an array with a single name key (for Pure enums) or
an array with both name and value keys
(for Backed enums) is created. All other cast types will result in an error.

[php_035] PHP - Modern OOP - Interfaces & Traits

Object Interfaces (implements, Multiple interfaces, Constants): Object Interfaces

Sumber: ht

Object Interfaces
Object interfaces allow you to create code which specifies which methods and properties a class must implement, without having to define how these methods or properties are implemented. Interfaces share a namespace with classes, traits, and enumerations, so they may not use the same name.
Interfaces are defined in the same way as a class, but with the interface keyword replacing the class keyword and without any of the methods having their contents defined.
All methods declared in an interface must be public; this is the nature of an interface.
In practice, interfaces serve two complementary purposes:
To allow developers to create objects of different classes that may be used interchangeably because they implement the same interface or interfaces. A common example is multiple database access services, multiple payment gateways, or different caching strategies. Different implementations may be swapped out without requiring any changes to the code that uses them.
To allow a function or method to accept and operate on a parameter that conforms to an interface, while not caring what else the object may do or how it is implemented. These interfaces are often named like Iterable , Cacheable , Renderable , and so on to describe the significance of the behavior.
Interfaces may define magic methods to require implementing classes to implement those methods.
Although they are supported, including constructors in interfaces is strongly discouraged. Doing so significantly reduces the flexibility of the object implementing the interface. Additionally, constructors are not enforced by inheritance rules, which can cause inconsistent and unexpected behavior.

[php_036] PHP - Modern OOP - Interfaces & Traits

Object Interfaces (implements, Multiple interfaces, Constants): implements

Sumber: ht

implements
To implement an interface, the implements operator is used.
All methods in the interface must be implemented within a class; failure to do so will result in a fatal error. Classes may implement more than one interface if desired by separating each interface with a comma.
A class that implements an interface may use a different name for its parameters than the interface. However, as of PHP 8.0 the language supports named arguments , which means callers may rely on the parameter name in the interface. For that reason, it is strongly recommended that developers use the same parameter names as the interface being implemented.
Interfaces can be extended like classes using the extends operator.
The class implementing the interface must declare all methods in the interface with a compatible signature . A class can implement multiple interfaces which declare a method with the same name. In this case, the implementation must follow the signature compatibility rules for all the interfaces. So covariance and contravariance can be applied.

[php_037] PHP - Modern OOP - Interfaces & Traits

Object Interfaces (implements, Multiple interfaces, Constants): Constants

Sumber: ht

Constants
It's possible for interfaces to have constants. Interface constants work exactly
like class constants .
Prior to PHP 8.1.0, they cannot be overridden by a class/interface that inherits them.

[php_038] PHP - Modern OOP - Interfaces & Traits

Object Interfaces (implements, Multiple interfaces, Constants): Properties

Sumber: ht

Properties
As of PHP 8.4.0, interfaces may also declare properties.
If they do, the declaration must specify if the property is to be readable,
writeable, or both.
The interface declaration applies only to public read and write access.
A class may satisfy an interface property in multiple ways.
It may define a public property.
It may define a public
virtual property
that implements only the corresponding hook.
Or a read property may be satisfied by a readonly property.
However, an interface property that is settable may not be readonly .
Interface properties example

Contoh Kode / Perintah CLI:

```
strtoupper($this->writeable); }

    // The interface only requires the property be settable,
    // but also including get operations is entirely valid.
    // This example creates a virtual property, which is fine.
    public string $writeable {
        get => $this->written;
        set {
            $this->written = $value;
        }
    }

    // This property requires both read and write be possible,
    // so we need to either implement both, or allow it to have
    // the default behavior.
    public string $both {
        get => $this->all;
        set {
            $this->all = strtoupper($value);
        }
    }
}
?>
```

[php_039] PHP - Modern OOP - Interfaces & Traits

Object Interfaces (implements, Multiple interfaces, Constants): Interface example

Sumber: ht

&reftitle.examples;
Interface example
Extendable Interfaces
Variance compatibility with multiple interfaces
Multiple interface inheritance
Interfaces with constants
Interfaces with abstract classes
Extending and implementing simultaneously
An interface, together with type declarations, provides a good way to make sure
that a particular object contains particular methods. See
instanceof operator and
type declarations .

Contoh Kode / Perintah CLI:

```
vars[$name] = $var;
    }

    public function getHtml($template)
    {
        foreach($this->vars as $name => $value) {
            $template = str_replace('{' . $name . '}', $value, $template);
        }

        return $template;
    }
}

// This will not work
// Fatal error: Class BadTemplate contains 1 abstract methods
// and must therefore be declared abstract (Template::getHtml)
class BadTemplate implements Template
{
    private $vars = [];

    public function setVariable($name, $var)
    {
        $this->vars[$name] = $var;
    }
}
?>
```

[php_040] PHP - Modern Attributes Syntax

Attributes (Metadata reflection, #[Attribute] syntax): Attributes overview

Sumber: ht

Attributes overview
PHP attributes provide structured, machine-readable metadata for classes, methods,
functions, parameters, properties, and constants. They can be inspected at runtime
via the Reflection API , enabling dynamic
behavior without modifying code. Attributes provide a declarative way to annotate
code with metadata.
Attributes enable the decoupling of a feature's implementation from its usage. While
interfaces define structure by enforcing methods, attributes provide metadata across multiple

elements, including methods, functions, properties, and constants. Unlike interfaces, which enforce method implementations, attributes annotate code without altering its structure. Attributes can complement or replace optional interface methods by providing metadata instead of enforced structure. Consider an ActionHandler interface that represents an operation in an application. Some implementations may require a setup step while others do not. Instead of forcing all classes implementing ActionHandler to define a setUp() method, an attribute can indicate setup requirements. This approach increases flexibility, allowing attributes to be applied multiple times when necessary. Implementing optional methods of an interface with Attributes

Contoh Kode / Perintah CLI:

```
fileName)) {
            throw new RuntimeException("File does not exist");
        }
    }

    #[SetUp]
    public function targetDirectoryExists()
    {
        if (!file_exists($this->targetDirectory)) {
            mkdir($this->targetDirectory);
        } elseif (!is_dir($this->targetDirectory)) {
            throw new RuntimeException("Target directory $this->targetDirectory is not a directory");
        }
    }

    public function execute()
    {
        copy($this->fileName, $this->targetDirectory . '/' . basename($this->fileName));
    }
}

function executeAction(ActionHandler $actionHandler)
{
    $reflection = new ReflectionObject($actionHandler);

    foreach ($reflection->getMethods() as $method) {
        $attributes = $method->getAttributes(SetUp::class);

        if (count($attributes) > 0) {
            $methodName = $method->getName();

            $actionHandler->$methodName();
        }
    }

    $actionHandler->execute();
}

$copyAction = new CopyFile();
$copyAction->fileName = "/tmp/foo.jpg";
$copyAction->targetDirectory = "/home/user";

executeAction($copyAction);
```

[php_041] PHP - Modern Attributes Syntax

Attributes (Metadata reflection, #[Attribute] syntax): Attribute syntax

Sumber: ht

Attribute syntax
Attribute syntax consists of several key components. An attribute
declaration starts with #[and ends with
] . Inside, one or more attributes can be listed,
separated by commas. The attribute name can be unqualified, qualified,
or fully-qualified, as described in Using Namespaces Basics .
Arguments to the attribute are optional and enclosed in parentheses
() . Arguments can only be literal values or constant
expressions. Both positional and named argument syntax are supported.
Attribute names and their arguments are resolved to a class, and the arguments
are passed to its constructor when an instance of the attribute is requested
through the Reflection API. Therefore, it is recommended to introduce a class
for each attribute.
Attribute Syntax

Contoh Kode / Perintah CLI:

```
value = $value;
    }
}

// b.php

namespace Another;

use MyExample\MyAttribute;

#[MyAttribute]
#[\MyExample\MyAttribute]
#[MyAttribute(1234)]
#[MyAttribute(value: 1234)]
#[MyAttribute(MyAttribute::VALUE)]
#[MyAttribute(array("key" => "value"))]
#[MyAttribute(100 + 200)]
class Thing
{
}

#[MyAttribute(1234), MyAttribute(5678)]
class AnotherThing
{
}
```

[php_042] PHP - Modern Attributes Syntax

Attributes (Metadata reflection, #[Attribute] syntax): Reading Attributes with the Reflection API

Sumber: ht

Reading Attributes with the Reflection API
To access attributes from classes, methods, functions, parameters, properties,
and class constants, use the getAttributes method provided
by the Reflection API. This method returns an array of ReflectionAttribute
instances. These instances can be queried for the attribute name, arguments, and
can be used to instantiate an instance of the represented attribute.

Separating the reflected attribute representation from its actual instance provides more control over error handling, such as missing attribute classes, mistyped arguments, or missing values. Objects of the attribute class are instantiated only after calling ReflectionAttribute::newInstance , ensuring that argument validation occurs at that point.

Reading Attributes using Reflection API

&example.outputs;

int(1234)

}

object(MyAttribute)#3 (1) {

["value"]=>

int(1234)

}

]]>

Instead of iterating over all attributes on the reflection instance, you can retrieve only those of a specific attribute class by passing the attribute class name as an argument.

Reading Specific Attributes using Reflection API

Contoh Kode / Perintah CLI:

```
value = $value;
    }
}

#[MyAttribute(value: 1234)]
class Thing
{
}

function dumpAttributeData($reflection) {
    $attributes = $reflection->getAttributes();

    foreach ($attributes as $attribute) {
       var_dump($attribute->getName());
       var_dump($attribute->getArguments());
       var_dump($attribute->newInstance());
    }
}

dumpAttributeData(new ReflectionClass(Thing::class));

// ----------------------------------------

value = $value;
    }
}

#[MyAttribute(value: 1234)]
class Thing
{
}

function dumpMyAttributeData($reflection) {
    $attributes = $reflection->getAttributes(MyAttribute::class);

    foreach ($attributes as $attribute) {
       var_dump($attribute->getName());
       var_dump($attribute->getArguments());
       var_dump($attribute->newInstance());
```

```
    }
}

dumpMyAttributeData(new ReflectionClass(Thing::class));
```

[php_043] PHP - Modern Attributes Syntax

Attributes (Metadata reflection, #[Attribute] syntax): Declaring Attribute Classes

Sumber: ht

Declaring Attribute Classes
It is recommended to define a separate class for each attribute. In the simplest case, an empty class with the #[Attribute] declaration is sufficient.
The attribute can be imported from the global namespace using a use statement.
Simple Attribute Class
To restrict the types of declarations an attribute can be applied to, pass a bitmask as the first argument to the #[Attribute] declaration.
Using target specification to restrict where attributes can be used
Declaring MyAttribute on another type will now throw an exception during the call to ReflectionAttribute::newInstance
The following targets can be specified:
Attribute::TARGET_CLASS
Attribute::TARGET_FUNCTION
Attribute::TARGET_METHOD
Attribute::TARGET_PROPERTY
Attribute::TARGET_CLASS_CONSTANT
Attribute::TARGET_PARAMETER
Attribute::TARGET_ALL
By default, an attribute can only be used once per declaration. To allow an attribute to be repeatable, specify it in the bitmask of the #[Attribute] declaration using the Attribute::IS_REPEATABLE flag.
Using IS_REPEATABLE to allow attribute on a declaration multiple times

[php_044] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): SensitiveParameter Attribute

Sumber: ht

SensitiveParameter Attribute
Added the #[\SensitiveParameter] attribute to redact sensitive data in backtraces.

[php_045] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): error_log_mode INI directive

Sumber: ht

error_log_mode INI directive
The error_log_mode INI directive has been added which allows setting the permissions for the error log file.

[php_046] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): Enumerations properties in constant expressions

Sumber: ht

Enumerations properties in constant expressions
It is now possible to fetch properties of
Enumerations
in constant expressions.

[php_047] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): Type System Improvements

Sumber: ht

Type System Improvements
It is now possible to use null and false
as stand-alone types.
The true type has been added.
It is now possible to combine intersection and union types.
The type needs to be written in DNF .

[php_048] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): Constants in Traits

Sumber: ht

Constants in Traits
It is now possible to define constants in traits.

[php_049] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): Readonly Classes

Sumber: ht

Readonly Classes
Support for readonly
on classes has been added.

[php_050] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): cURL

Sumber: ht

cURL
Added the CURLINFO_EFFECTIVE_METHOD option,
which returns the effective HTTP method in the return value of
curl_getinfo .
Exposed multiple new constants from libcurl 7.62 to 7.80.
Added the curl_upkeep function to perform any connection upkeep checks.

[php_051] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): DBA

Sumber: ht

DBA
The LMDB Driver now accepts the DBA_LMDB_USE_SUB_DIR or
DBA_LMDB_NO_SUB_DIR flags to determine if it should
create a subdirectory or not when creating a database file.

[php_052] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): OCI8

Sumber: ht

OCI8
Added the oci8.prefetch_lob_size INI directive and
oci_set_prefetch_lob function to tune LOB query
performance by reducing the number of round-trips between PHP and
Oracle Databases when fetching LOBS.
This is usable with Oracle Database 12.2 or later.

[php_053] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): OpenSSL

Sumber: ht

OpenSSL
Added AEAD support for the chacha20-poly1305 algorithm.

[php_054] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): ODBC

Sumber: ht

ODBC
Added the odbc_connection_string_is_quoted ,
odbc_connection_string_should_quote , and
odbc_connection_string_quote functions.
These are primarily used behind the scenes in the ODBC and PDO_ODBC extensions,
but are exposed to userland for easier unit testing, and for user
applications and libraries to perform quoting themselves.

[php_055] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): PCRE

Sumber: ht

PCRE
Added support for the n (NO_AUTO_CAPTURE) modifier,
which makes simple (xyz) groups non-capturing.
Only named groups like (?<name>xyz) are capturing.
This only affects which groups are capturing, it is still possible to use

numbered subpattern references, and the matches array will still contain
numbered results.

[php_056] PHP - Modern Features - PHP 8.2

PHP 8.2 New Features (Readonly classes, null/false as standalone types, DNF types): Random

Sumber: ht

Random
This is a new extension which organises and consolidates existing
implementations related to random number generators.
New and better RNGs are available with scope issues eliminated.

[php_057] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Readonly Amendments

Sumber: ht

Readonly Amendments
Anonymous classes may now be marked as readonly.
Readonly properties can now be reinitialized during cloning.

[php_058] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Typed Class Constants

Sumber: ht

Typed Class Constants
Class, interface, trait, and enum constants now support type declarations.

[php_059] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Closures created from magic methods

Sumber: ht

Closures created from magic methods
Closures created from magic methods can now accept named arguments.

[php_060] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): The final modifier with a method from a trait

Sumber: ht

The final modifier with a method from a trait
The final modifier may now be used when using a method from a trait.

[php_061] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Override Attribute

Sumber: ht

Override Attribute
Added the #[\Override] attribute to check that a method exists in a parent
class or implemented interface.

[php_062] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Fetch class constant dynamically syntax

Sumber: ht

Fetch class constant dynamically syntax
Class constants can now be accessed dynamically using the
C::{$name} syntax.

[php_063] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Static variable Initializers

Sumber: ht

Static variable Initializers
Static variable initializers can now contain arbitrary expressions.

[php_064] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Fallback value syntax for ini variables

Sumber: ht

Fallback value syntax for ini variables
php.ini now supports fallback/default value syntax.

[php_065] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): CLI

Sumber: ht

CLI
It is now possible to lint multiple files.

[php_066] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): DOM

Sumber: ht

DOM
Added properties DOMElement::$className
and DOMElement::$id .
These are not binary-safe at the moment because of underlying limitations
of libxml2.
This means that the property values will be cut off at a NUL byte.
Added properties DOMNode::$isConnected
and DOMNameSpaceNode::$isConnected .
Added properties DOMNode::$parentElement

and DOMNameSpaceNode::$parentElement .

[php_067] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): FFI

Sumber: ht

FFI
It is now possible to assign FFI\CData to other FFI\CData.
Meaning CData can now be assigned to structs and fields.

[php_068] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Opcache

Sumber: ht

Opcache
opcache_get_status()['scripts'][n]['revalidate'] now contains
a Unix timestamp of when the next revalidation of the scripts timestamp is
due, dictated by the
opcache.revalidate_freq
INI directive.

[php_069] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): POSIX

Sumber: ht

POSIX
posix_getrlimit now takes an optional
resource parameter to allow fetching a single resource limit.
posix_isatty now raises type warnings for integers
following the usual ZPP semantics.
posix_ttyname now raises type warnings for integers
following the usual ZPP semantics and value warnings for invalid file
descriptor integers.

[php_070] PHP - Modern Features - PHP 8.3

PHP 8.3 New Features (Typed class constants, json_validate(), #[\Override]): Streams

Sumber: ht

Streams
Streams can now emit the STREAM_NOTIFY_COMPLETED
notification. This was previously not implemented.

[php_071] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Property Hooks

Sumber: ht

Property Hooks
Object properties may now have additional logic associated with their
get and set operations.
Depending on the usage, that may or may not make the property virtual,
that is, it has no backing value at all.

Contoh Kode / Perintah CLI:

```
$this->firstName . ' ' . $this->lastName;
    }

    // All write operations go through this hook, and the result is what is written.
    // Read access happens normally.
    public string $firstName {
        set => ucfirst(strtolower($value));
    }

    // All write operations go through this hook, which has to write to the backing value itself.
    // Read access happens normally.
    public string $lastName {
        set {
            if (strlen($value) lastName = $value;
        }
    }
}

$p = new Person();

$p->firstName = 'peter';
print $p->firstName; // Prints "Peter"
$p->lastName = 'Peterson';
print $p->fullName; // Prints "Peter Peterson"

$p->fullName = "Peter 'Pete' Peterson"; // Throws Error: "Property Person::$fullName is read-only"
```

[php_072] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Asymmetric Property Visibility

Sumber: ht

Asymmetric Property Visibility
Object properties may now have their set visibility
controlled separately from the get visibility.

Contoh Kode / Perintah CLI:

```
name = $name;
    }
}
```

[php_073] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Lazy Objects

Sumber: ht

Lazy Objects
It is now possible to create objects whose initialization is deferred until
they are accessed. Libraries and frameworks can leverage these lazy objects
to defer fetching data or dependencies required for initialization.

Contoh Kode / Perintah CLI:

```
__construct($data);
};

$reflector = new ReflectionClass(Example::class);
$object = $reflector->newLazyGhost($initializer);
```

[php_074] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): #[\Deprecated] attribute

Sumber: ht

#[\Deprecated] attribute
The new Deprecated attribute can be used to mark functions, methods,
and class constants as deprecated. The behavior of functionality deprecated with this
attribute matches the behavior of the existing deprecation mechanism for functionality
provided by PHP itself. The only exception is that the emitted error code is
E_USER_DEPRECATED instead of E_DEPRECATED .
Existing deprecations in functionality provided by PHP itself have been updated to use
the attribute, improving the emitted error messages by including a short explanation.

[php_075] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Parsing RFC1867 (multipart) requests in non-POST HTTP requests

Sumber: ht

Parsing RFC1867 (multipart) requests in non-POST HTTP requests
Added request_parse_body function that allows parsing
RFC1867 (multipart) requests in non-POST HTTP requests.

[php_076] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Chaining &new; expressions without parentheses

Sumber: ht

Chaining &new; expressions without parentheses
New expressions with constructor arguments are now dereferencable, meaning
they allow chaining method calls, property accesses, etc. without enclosing
the expression in parentheses.

[php_077] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Improved Debugging Info for WeakReference

Sumber: ht

Improved Debugging Info for WeakReference
Getting the debug info for WeakReference will now
also output the object it references, or &null; if the reference is no
longer valid.

[php_078] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Improved Debugging Info for Closure

Sumber: ht

Improved Debugging Info for Closure
The output of Closure::__debugInfo now includes
the name, file, and line of the Closure .

[php_079] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Defining Identical Symbols in Different Namespace Blocks

Sumber: ht

Defining Identical Symbols in Different Namespace Blocks
Exiting a namespace now clears seen symbols.
This allows using a symbol in a namespace block, even if a previous
namespace block declared a symbol with the same name.

[php_080] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): cURL

Sumber: ht

cURL
curl_version returns an additional
feature_list value, which is an associative array
of all known cURL features, and whether they are supported (&true;)
or not (&false;).
Added CURL_HTTP_VERSION_3 and
CURL_HTTP_VERSION_3ONLY constants (available
since libcurl 7.66 and 7.88) as available options for
CURLOPT_HTTP_VERSION .
Added CURLOPT_PREREQFUNCTION as a cURL option that
accepts a callable to be called after the connection is made,
but before the request is sent.
This callable must return either CURL_PREREQFUNC_OK or
CURL_PREREQFUNC_ABORT to allow or abort the request.
Added CURLOPT_SERVER_RESPONSE_TIMEOUT ,
which was formerly known as CURLOPT_FTP_RESPONSE_TIMEOUT .
Both constants hold the same value.
Added CURLOPT_DEBUGFUNCTION as a cURL option that
accepts a callable that gets called during the request lifetime
with the CurlHandle object,
an integer containing the debug message type, and a string containing the
debug message.
The debug message type is one of the following constants:
CURLINFO_TEXT
CURLINFO_HEADER_IN
CURLINFO_HEADER_OUT
CURLINFO_DATA_IN
CURLINFO_DATA_OUT
CURLINFO_SSL_DATA_IN
CURLINFO_SSL_DATA_OUT
Once this option is set, CURLINFO_HEADER_OUT
must not be set because it uses the same libcurl functionality.
The curl_getinfo now returns an additional
posttransfer_time_us key, containing the number of
microseconds from the start until the last byte is sent.
When a redirect is followed, the time from each request is added together.
This value can also be retrieved by passing

CURLINFO_POSTTRANSFER_TIME_T to the
curl_getinfo option parameter.
This requires libcurl 8.10.0 or later.

[php_081] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): DOM

Sumber: ht

DOM
Added the Dom namespace with new classes as counterparts
to the existing DOM classes (e.g. Dom\Node is the new
DOMNode).
These classes are compatible with HTML 5 and are WHATWG spec-compliant;
solving long-standing bugs in the DOM extension.
The old DOM classes remain available for backwards compatibility.
Added the DOMNode::compareDocumentPosition
with its associated constants:
DOMNode::DOCUMENT_POSITION_DISCONNECTED
DOMNode::DOCUMENT_POSITION_PRECEDING
DOMNode::DOCUMENT_POSITION_FOLLOWING
DOMNode::DOCUMENT_POSITION_CONTAINS
DOMNode::DOCUMENT_POSITION_CONTAINED_BY
DOMNode::DOCUMENT_POSITION_IMPLEMENTATION_SPECIFIC
It is now possible to pass any callable to
DOMXPath::registerPhpFunctions .
Furthermore, with DOMXPath::registerPhpFunctionNs ,
callbacks can now be registered that will use native function call syntax
rather than using php:function('name') .

[php_082] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Intl

Sumber: ht

Intl
Added the NumberFormatter::ROUND_HALFODD to
complement the existing NumberFormatter::ROUND_HALFEVEN
functionality.

[php_083] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): OpenSSL

Sumber: ht

OpenSSL
Added support for Curve25519 + Curve448 based keys.
Specifically x25519, ed25519, x448 and ed448 fields are supported in
openssl_pkey_new ,
openssl_pkey_get_details ,
openssl_sign , and
openssl_verify were extended to support those keys.

Implement PASSWORD_ARGON2 password hashing.
Requires OpenSSL 3.2 and NTS build.

[php_084] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): PCRE

Sumber: ht

PCRE
The bundled pcre2lib has been updated to version 10.44.
As a consequence, LoongArch JIT support has been added, spaces
are now allowed between braces in Perl-compatible items, and
variable-length lookbehind assertions are now supported.
With pcre2lib version 10.44, the maximum length of named capture groups
has changed from 32 to 128 .
Added support for the r (PCRE2_EXTRA_CASELESS_RESTRICT)
modifier, as well as the (?r) mode modifier.
When enabled along with the case-insensitive modifier (i),
the expression locks out mixing of ASCII and non-ASCII characters.

[php_085] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): PDO

Sumber: ht

PDO
Added support for driver-specific subclasses in order to better support
database-specific functionalities.
The new classes are instantiatable either via calling the
PDO::connect method or by instantiating one
of the driver-specific subclasses directly.
Added support for driver specific SQL parsers.
When a driver-specific parser is not available, the default parser is used.
The default parser supports:
single and double-quoted literals, with doubling as escaping mechanism
two-dashes and non-nested C-style comments

[php_086] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): PDO_MYSQL

Sumber: ht

PDO_MYSQL
Added a custom parser supporting:
single and double-quoted literals, with doubling and backslash as escaping
mechanism
backtick literal identifiers and with doubling as escaping mechanism
two dashes followed by at least 1 whitespace, non-nested C-style comments,
and hash-comments

[php_087] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): PDO_PGSQL

Sumber: ht

PDO_PGSQL
Added a custom parser supporting:
single and double-quoted literals, with doubling as escaping mechanism
C-style "escape" string literals (E'string')
dollar-quoted string literals
two-dashes and C-style comments (non-nested)
support for ?? as escape sequence for the
? operator

[php_088] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): PDO_SQLITE

Sumber: ht

PDO_SQLITE
Added a custom parser supporting:
single, double-quoted, and backtick literals, with doubling as
escaping mechanism
square brackets quoting for identifiers
two-dashes and C-style comments (non-nested)

[php_089] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Phar

Sumber: ht

Phar
Added support for the Unix timestamp extension for Zip archives.

[php_090] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Readline

Sumber: ht

Readline
Added ability to change the .php_history path through
the PHP_HISTFILE environment variable.

[php_091] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Reflection

Sumber: ht

Reflection
ReflectionAttribute now contains a
name property to improve the debugging experience.
ReflectionClassConstant::__toString and
ReflectionProperty::__toString now returns the

attached doc comments.
Multiple new methods and constants which are related to the lazy objects
feature have been added:
ReflectionClass::newLazyGhost
ReflectionClass::newLazyProxy
ReflectionClass::resetAsLazyGhost
ReflectionClass::resetAsLazyProxy
ReflectionClass::isUninitializedLazyObject
ReflectionClass::initializeLazyObject
ReflectionClass::markLazyObjectAsInitialized
ReflectionClass::getLazyInitializer
ReflectionProperty::skipLazyInitialization
ReflectionProperty::setRawValueWithoutLazyInitialization
ReflectionClass::SKIP_INITIALIZATION_ON_SERIALIZE
ReflectionClass::SKIP_DESTRUCTOR

[php_092] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): SOAP

Sumber: ht

SOAP
Added support for clark notation for namespaces in class map.
It is now possible to specify entries in a class map with clark notation
to resolve a type with a specific namespace to a specific class.
For example: '{http://example.com}foo' => 'FooClass' .
Instances of DateTimeInterface that are
passed to xsd:datetime or similar elements are now
serialized as such instead of being serialized as an empty string.
Session persistence now works with a shared session module.

[php_093] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Standard

Sumber: ht

Standard
-->
Added a new RoundingMode enum with clearer naming
and improved discoverability compared to the
PHP_ROUND_* constants.
Moreover, four new rounding modes were added which are only available via
the new RoundingMode enum.

[php_094] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): XSL

Sumber: ht

XSL
It is now possible to use parameters that contain both single and double
quotes.

It is now possible to pass any callable to
XSLTProcessor::registerPhpFunctions .
Added XSLTProcessor::$maxTemplateDepth and
XSLTProcessor::$maxTemplateVars
to control the recursion depth of XSL template evaluation.

[php_095] PHP - Modern Features - PHP 8.4

PHP 8.4 New Features (Property Hooks, Asymmetric Visibility, new without parentheses): Zip

Sumber: ht

Zip
Added the ZipArchive::ER_TRUNCATED_ZIP
constant, which was added in libzip 1.11.